

Strategies for learning $O(h_i)$ token continuations

1 Use mini-batch SGD to learn $Alice(S, O)$.

So S and O are jointly learned.

2 Use mini-batch SGD to learn $Alice_{cila}(S, O, O')$.

So S , O and O' are jointly learned.

3 Pre-train a single layer $Alice(S, O_1)$ to learn a spine $\{h_i\}$.

Hold this spine fixed and from an L -layer O learn $Alice$ ribs $O(h_i)$.

So S and O are learned in stages.

4 Pre-train an $Ecila(S', O')$ to learn a spine $\{h_i\}$, discarding $\{o'_i\}$.

Hold this spine fixed and learn $Alice$ ribs $O(h_i)$.

So hybrid state machine (S', O) , serving two masters, is learned in stages.

Each of these learning strategies represents a different utilization of the word embedding and tradeoff between training problem faithfulness and time.

Strategy 1 is parsimonious.

Strategy 2 utilizes the most word embedding information, but is also the most time expensive.

Strategy 3 is faster than 1, but suboptimal in solving the $Alice$ training problem.

Hybrid strategy 4 is faster than 3 and suboptimal to 2.

Burn-in

$Alice$ is a state machine wherein the initial state is arbitrary or set to a default (like all zeros) that doesn't yet reflect meaningful history. In practice during this initial burn-in period, the early outputs being based on little data are deliberately discarded or ignored, until training problem metrics (here perplexity and entropy) begin to settle down as sufficient experience accumulates, at which point we reset the initial state to the current state.

Alice is a LLM designed for users to experience conversations more grounded in meaning with fewer hallucinations.